

Aula 13 - Testes unitários, de integração e de sistema

TJSC 2026 - Analista de Sistemas - Banca FGV

Recorte da aula

Esta aula cobre o item 1.16 do edital: testes de software unitário, integração e sistema. O objetivo é transformar sua vivência com Java, Quarkus, Angular, APIs e esteiras corporativas em resposta de prova: distinguir níveis de teste, tipos de teste, técnicas, artefatos e armadilhas conceituais que a FGV costuma explorar.

1. Como a FGV costuma cobrar testes de software

Em Engenharia de Software, a FGV costuma formular questões de testes com três estratégias: definição precisa, cenário de equipe/projeto e comparação entre termos próximos. Em vez de perguntar apenas 'o que é teste unitário', a banca tende a apresentar um sistema com módulos, APIs, banco de dados, telas ou regras de negócio e pedir que o candidato classifique o nível de teste ou a finalidade do procedimento.

Para esta aula, foram usados como calibração itens oficiais recentes da FGV sobre testes unitários, integração, sistema, regressão, estresse, análise estática e ferramentas de qualidade. A base técnica foi cruzada com glossários e normas de referência, especialmente ISTQB, ISO/IEC/IEEE 29119 e documentação oficial de tecnologias usadas no seu contexto, como JUnit, Quarkus e Angular.

Ponto de prova

Não confunda nível de teste com tipo de teste. Unitário, integração e sistema são níveis. Regressão, desempenho, segurança, usabilidade, carga e estresse são tipos ou objetivos de teste que podem ocorrer em diferentes níveis, embora alguns apareçam mais frequentemente em nível de sistema.

2. Fundamentos: teste, verificação, validação, erro, defeito e falha

Teste de software é uma atividade de avaliação executada para revelar defeitos, fornecer informação sobre qualidade e aumentar confiança no produto. Para prova, teste não prova ausência de defeitos; ele reduz incerteza e evidencia comportamentos sob condições planejadas.

Termo	Ideia central	Como a FGV pode inverter
Erro	Ação humana incorreta: interpretação errada da regra, implementação equivocada, especificação incompleta.	Trocar erro por defeito no código.
Defeito	Problema existente no artefato: código, requisito, configuração, script, caso de teste ou documento.	Dizer que defeito é sempre execução incorreta. Defeito pode estar latente.
Falha	Manifestação observável do defeito durante a execução, quando o sistema não se comporta como esperado.	Dizer que falha é sinônimo de bug armazenado no repositório.
Verificação	Confere se o produto foi construído corretamente em relação à especificação: 'estamos construindo certo?'.	Confundir com validação junto ao usuário.
Validação	Confere se o produto atende à necessidade real: 'estamos construindo a coisa certa?'.	Reduzir validação a teste unitário técnico.

Em sistemas corporativos, um exemplo simples é a regra de cálculo de tarifas. Se o analista entendeu incorretamente a regra, há erro humano. Se essa regra foi codificada de modo incorreto no serviço Java, há defeito. Se, em produção ou homologação, a API retorna valor indevido, há falha observável.

3. Níveis de teste: unitário, integração e sistema

A cobrança central desta aula está nos níveis de teste. Nível de teste é uma organização de atividades com escopo e objetivo definidos. A distinção que mais cai é: unidade isolada, interação entre unidades e comportamento do sistema como um todo.

Nível	Escopo	Objetivo	Exemplo em Java/Quarkus/Angular	Armadilha FGV
Teste unitário	Menor unidade testável: método, classe, função, componente lógico.	Verificar comportamento isolado, com dependências controladas por stubs, mocks ou fakes quando necessário.	Testar um método de cálculo de desconto de um service Java sem subir HTTP, banco ou mensageria.	Chamar de unitário um teste que sobe aplicação completa, banco e endpoint real.
Teste de integração	Interação entre módulos, componentes, serviços, banco, fila, API ou camadas.	Verificar se partes combinadas se comunicam corretamente.	Testar uma rota Quarkus chamando repository, banco de teste e serialização JSON; ou componente Angular interagindo com service mockado/real.	Confundir integração com sistema apenas porque há mais de uma classe envolvida.
Teste de sistema	Sistema completo ou subsistema integrado em ambiente representativo.	Avaliar comportamento fim a fim frente a requisitos funcionais e não funcionais.	Executar fluxo de login, consulta, autorização, persistência e auditoria em ambiente de homologação.	Confundir com aceitação: sistema é técnico; aceitação foca decisão de aceitar pelo usuário/cliente.

Regra de memorização

Unidade = parte isolada. Integração = partes conversando. Sistema = produto integrado como um todo. Aceitação = cliente/usuário decide se atende ao negócio. O edital cobra unitário, integração e sistema; aceitação deve ser conhecida para não cair em pegadinha.

4. Teste unitário: isolamento, assertiva e determinismo

Teste unitário deve ser pequeno, rápido, determinístico e focado em uma unidade. Ele é excelente para regras de negócio puras, validações, transformações, cálculos, parsing, formatação, políticas de autorização e comportamento de classes com dependências substituíveis.

A estrutura clássica é Arrange, Act, Assert: preparar dados e dependências; executar o comportamento; verificar resultado e efeitos esperados. A FGV pode não usar essa nomenclatura, mas explora a ideia de precondições, entrada, execução e resultado esperado.

```
class CalculadoraTaxa {
    BigDecimal calcular(BigDecimal valor, boolean clienteServidorPublico) {
        if (clienteServidorPublico) {
            return valor.multiply(new BigDecimal("0.01"));
        }
        return valor.multiply(new BigDecimal("0.02"));
    }
}

@Test
void deveAplicarTaxaReduzidaParaServidorPublico() {
    var calc = new CalculadoraTaxa();
    var taxa = calc.calcular(new BigDecimal("1000.00"), true);
    assertEquals(new BigDecimal("10.0000"), taxa);
}
```

O exemplo acima é unitário porque não depende de rede, banco, contêiner, autenticação, fila, endpoint HTTP ou ambiente externo. Em prova, se o enunciado disser que a equipe mockou as dependências e testou apenas a regra de uma classe, a resposta tende a ser teste unitário.

Boa prática	Por quê	Possível pegadinha
Isolar dependências externas	Reduz instabilidade e torna o teste rápido.	Isolar não significa testar sem assertiva ou sem critério de sucesso.
Usar assertivas objetivas	O teste precisa declarar resultado esperado.	Executar código sem assertiva tende a ser teste fraco.
Evitar dependência de hora, rede e ordem global	Garante determinismo .	Teste que falha aleatoriamente em CI/CD é flaky , não necessariamente prova defeito funcional.
Cobrir regra relevante, não implementação trivial	Evita teste frágil e pouco informativo.	100% de cobertura não garante ausência de defeitos.

5. Test doubles: dummy, stub, fake, spy e mock

Em testes unitários, frequentemente substituímos dependências reais por objetos controlados. A FGV pode explorar termos próximos; por isso, a distinção é importante.

Termo	Função	Exemplo
Dummy	Objeto passado apenas para preencher parâmetro; não é usado de forma relevante.	Um <code>usuarioLogado</code> mínimo passado a um método que não acessa seus campos.
Stub	Retorna respostas pré-definidas para permitir o teste.	<code>RepositorioClienteStub</code> retorna cliente ativo quando chamado.
Fake	Implementação funcional simplificada, geralmente em memória.	<code>Repository</code> em memória para simular banco.
Spy	Registra chamadas para posterior verificação, podendo delegar comportamento real.	Verificar que <code>notificador.enviar()</code> foi chamado uma vez.
Mock	Objeto configurado com expectativas de interação; valida chamadas e argumentos.	Mock de gateway externo esperando chamada com CPF e valor.

Pegadinha comum

Stub responde. Mock verifica interação. Fake implementa de forma simplificada. Spy observa chamadas. Dummy só preenche espaço. Na prática as bibliotecas misturam nomenclaturas; em prova, considere a definição conceitual.

6. Teste de integração: interfaces, contratos e dependências reais ou controladas

Teste de integração avalia a **cooperação entre unidades**. O foco deixa de ser a lógica isolada e passa a ser a comunicação entre componentes: service e repository; API e camada de autenticação; frontend e service; aplicação e banco; produtor e consumidor de mensagem; endpoint e serialização JSON.

Em Quarkus, um teste com `@QuarkusTest`, chamada **HTTP via REST Assured**, serialização JSON, validação de status HTTP e uso de banco de teste tende a ser classificado como integração, ainda que o teste esteja automatizado e rode no pipeline. O fato de ser automatizado não torna o teste unitário.

```
@QuarkusTest
class ClienteResourceTest {
    @Test
    void deveBuscarClientePorId() {
        given()
        .when().get("/clientes/10")
        .then()
        .statusCode(200)
    }
}
```

```

        .body("id", is(10));
    }
}

```

No frontend Angular, um teste de componente que instancia template, injeta services e verifica renderização/interação pode ficar entre unitário de componente e integração de componente. Para prova, observe o foco: se testa apenas a classe do componente, tende a unitário; se envolve template, injeção, service e DOM, a alternativa pode apontar integração.

Forma de integração	O que verifica	Risco que pega
Camada de persistência	Mapeamento, query, transação, constraints, conversão de tipos.	Teste unitário com mock não revela erro de SQL real.
API interna	Contrato HTTP, status code, payload, autenticação e serialização.	Endpoint compila, mas responde schema incompatível.
Mensageria	Publicação, consumo, formato da mensagem e idempotência.	Ordem, duplicidade e retry mal tratados.
Frontend-backend	Campos, validações, contrato JSON, erro de mapeamento.	Mudança de DTO quebra tela mesmo com services isolados passando.

7. Teste de sistema: comportamento completo e ambiente representativo

Teste de sistema ocorre com o sistema integrado, avaliando requisitos funcionais e não funcionais em um ambiente representativo. Não significa necessariamente produção, mas deve ser ambiente suficientemente próximo para avaliar o comportamento global.

Exemplo: um sistema do Judiciário permite cadastro de processo, anexação de documento, autenticação, assinatura, tramitação e consulta. Um teste que percorre o fluxo completo, com regras de permissão, auditoria, banco, serviços e interface, é teste de sistema. Se o usuário final homologa e decide aceitar a solução, entra a ideia de teste de aceitação, que é conceito vizinho.

Aspecto	Teste de integração	Teste de sistema
Foco	Interação entre partes selecionadas.	Comportamento do sistema integrado como um todo.
Escopo	Pode envolver duas ou mais camadas ou serviços.	Abrange fluxos completos e requisitos globais.
Ambiente	Pode usar ambiente controlado com substituições.	Deve ser representativo do uso real.
Defeitos típicos	Contrato, comunicação, serialização, transação.	Regra fim a fim, usabilidade, desempenho, segurança, compatibilidade.

8. Níveis de teste versus tipos de teste

Nível responde 'em que escopo testamos?'. Tipo responde 'com que objetivo testamos?'. A FGV frequentemente mistura os dois para ver se o candidato sabe classificar.

Tipo de teste	Objetivo	Pode ocorrer em qual nível?
Funcional	Verificar funcionalidades especificadas.	Unitário, integração, sistema e aceitação.
Regressão	Verificar se mudanças não quebraram comportamento já existente.	Qualquer nível.
Desempenho	Avaliar tempo de resposta, vazão, recursos, escalabilidade.	Mais comum em sistema, mas pode haver medições em componentes.
Carga	Verificar comportamento sob carga esperada ou crescente.	Normalmente sistema.
Estresse	Expor o sistema a condições extremas/anormais.	Normalmente sistema.
Segurança	Identificar vulnerabilidades, autorização, autenticação e exposição de dados.	Integração e sistema, podendo ter testes unitários de regras.

Usabilidade	Avaliar facilidade de uso, eficiência e experiência.	Sistema/aceitação.
Compatibilidade	Avaliar navegadores, sistemas, versões, dispositivos.	Sistema.

FGV adora

Dizer que regressão é nível de teste é erro. Dizer que unitário, integração e sistema são tipos também é impreciso. O mais seguro: unitário, integração e sistema = níveis; regressão, desempenho, segurança, usabilidade, carga e estresse = tipos/objetivos.

9. Técnicas de projeto de testes

Mesmo quando o edital não lista técnicas formalmente, elas aparecem como **vocabulário de qualidade e engenharia**. Para prova, tenha o essencial: caixa-preta, caixa-branca, partição de equivalência, análise de valor-limite, tabela de decisão, transição de estados e cobertura estrutural.

Técnica	Ideia	Exemplo
Caixa-preta	Deriva testes da especificação, sem olhar estrutura interna.	Testar CPF válido/inválido por classes de entrada.
Caixa-branca	Deriva testes da estrutura interna do código.	Cobrir ramos if/else de método de autorização.
Partição de equivalência	Divide entradas em classes que tendem a ter comportamento semelhante.	Idade menor que 18, entre 18 e 65, maior que 65.
Valor-limite	Testa bordas das classes, onde há mais chance de erro.	17, 18, 19 para regra de maioridade.
Tabela de decisão	Usa combinações de condições e ações.	Perfil, status, pendência documental e tipo de autorização.
Transição de estados	Modela estados e eventos válidos/inválidos.	Processo: cadastrado, distribuído, concluso, arquivado.
Cobertura estrutural	Mede quanto do código foi exercitado.	Cobertura de instruções, ramos/decisões e caminhos.

Cobertura é métrica, não garantia absoluta. 100% de cobertura de instruções significa que todas as instruções mapeadas foram executadas por testes, mas não garante que todas as combinações, regras de negócio, valores-limite, efeitos colaterais e integrações foram verificadas corretamente.

10. Automação de testes e pirâmide de testes

Automação busca execução repetível, rápida e confiável. Em uma esteira CI/CD, testes unitários devem fornecer feedback rápido; testes de integração validam dependências relevantes; testes de sistema ou ponta a ponta tendem a ser mais lentos, custosos e sensíveis a ambiente.

A pirâmide de testes é um modelo de organização: muitos testes unitários na base, uma quantidade menor de testes de integração/serviço no meio e poucos testes de ponta a ponta/interface no topo. A FGV pode cobrar a razão econômica: testes mais amplos costumam ser mais lentos, caros e frágeis; testes menores são rápidos, mas não substituem a validação do sistema integrado.

Camada	Quantidade esperada	Características
Unitários	Maior quantidade	Rápidos, baratos, isolados, bons para regra de negócio.
Integração/serviço	Quantidade intermediária	Valida contratos, banco, APIs, mensageria, serialização.
Sistema/E2E	Menor quantidade	Fluxos completos, maior custo, maior valor de confiança fim a fim.

11. Diferença entre uso real no trabalho e cobrança FGV

No trabalho	Na prova FGV
O time chama tudo de 'teste automatizado' ou 'teste de API'.	A banca quer classificar nível, tipo, técnica e objetivo.
Um teste Quarkus pode rodar localmente como parte do Maven.	Se sobe contexto, endpoint e banco, provavelmente não é unitário puro.
Cobertura é usada como meta de quality gate.	Cobertura não prova qualidade nem ausência de defeitos.
Mocks são usados de forma pragmática.	A banca pode diferenciar mock, stub, fake e spy conceitualmente.
Testes de interface detectam problemas reais de integração visual.	A banca pode apontar que são mais lentos e frágeis que testes unitários.

12. Pegadinhas FGV de alto ROI

- Confundir teste de integração com teste de sistema: integração avalia interação entre partes; sistema avalia o sistema integrado como um todo.
- Chamar regressão de nível de teste: regressão é objetivo/tipo, pode ocorrer em qualquer nível.
- Afirmar que teste prova ausência de defeitos: teste aumenta confiança, mas não prova ausência.
- Afirmar que 100% de cobertura garante qualidade: cobertura mede execução de itens, não correção semântica completa.
- Confundir verificação e validação: verificação pergunta se construímos corretamente; validação pergunta se atendemos à necessidade real.
- Confundir teste de estresse com teste de carga: carga usa volume esperado ou crescente; estresse vai a condições extremas/anormais.
- Confundir teste unitário com teste automatizado: unitário é nível/escopo; automatizado é forma de execução.
- Afirmar que teste de sistema substitui testes unitários: cada nível revela defeitos diferentes.

13. Lista de questões

Responda às questões antes de consultar o gabarito. As questões são autorais, calibradas pelo estilo FGV, com enunciados contextualizados e distratores tecnicamente próximos.

1. Uma equipe do setor de tecnologia de um tribunal criou uma classe Java responsável por calcular o prazo final de manifestação processual a partir de uma data inicial, do tipo de prazo e da existência de feriado local. Para validar apenas essa regra, sem acionar banco de dados, endpoint HTTP, autenticação, fila ou serviço externo, os desenvolvedores instanciaram a classe diretamente e verificaram o resultado esperado por meio de assertivas. Nesse cenário, o nível de teste realizado é melhor classificado como

- A) teste de sistema, pois verifica uma regra relevante para o usuário final.
- B) teste unitário, pois exercita uma unidade de código em isolamento controlado.
- C) teste de integração, pois qualquer regra de negócio depende de requisitos.
- D) teste de aceitação, pois envolve prazo processual e interesse do usuário.
- E) teste de regressão, pois evita que a regra volte a falhar no futuro.

2. No contexto de uma API corporativa construída com Quarkus, determinada suíte de testes sobe o contexto da aplicação, chama endpoints REST reais, serializa e deserializa JSON, utiliza um banco de dados de teste e verifica se a camada de persistência grava corretamente os dados. Embora seja executada automaticamente no pipeline, essa suíte deve ser classificada, predominantemente, como teste

- A) unitário, pois é automatizada e escrita com JUnit.
- B) de aceitação, pois usa dados semelhantes aos de produção.

- C) de regressão, pois todo teste de pipeline tem finalidade regressiva.
- D) de integração, pois verifica a interação entre endpoint, serviço, serialização e persistência.
- E) estático, pois a correção é inferida antes da implantação em produção.

3. Considere as afirmativas sobre níveis e tipos de teste. I. Teste unitário concentra-se em unidades isoladas, como métodos, funções, classes ou componentes. II. Teste de integração verifica a interação entre componentes, módulos, serviços ou sistemas. III. Teste de regressão é um nível de teste hierarquicamente situado entre integração e sistema. Está correto o que se afirma em

- A) I, apenas.
- B) II, apenas.
- C) I e II, apenas.
- D) II e III, apenas.
- E) I, II e III.

4. Em uma revisão de qualidade, foram feitas as seguintes associações: (1) Stub; (2) Mock; (3) Fake; (4) Spy. () Implementação simplificada, muitas vezes em memória, que substitui uma dependência real. () Objeto que fornece respostas pré-definidas para o teste. () Objeto usado para verificar chamadas, argumentos e interações esperadas. () Objeto que registra interações para inspeção posterior. A sequência correta é

- A) 3 - 1 - 2 - 4.
- B) 1 - 2 - 3 - 4.
- C) 2 - 1 - 4 - 3.
- D) 4 - 3 - 2 - 1.
- E) 3 - 2 - 1 - 4.

5. Uma aplicação Angular possui um componente de consulta processual. Um teste instancia o componente com TestBed, injeta um serviço simulado, renderiza o template, simula digitação do usuário e verifica se a tabela exibe a mensagem correta após o retorno do serviço. Considerando o escopo descrito, a classificação mais adequada é

- A) teste estático, porque não há chamada real ao servidor.
- B) teste unitário puro, porque todo teste de componente Angular é unitário por definição.
- C) teste de carga, porque simula interação de usuário.
- D) teste de sistema, porque envolve tela e usuário.
- E) teste de componente com caráter de integração, pois envolve classe, template, injeção de dependência e interação controlada.

6. Sobre cobertura de testes, assinale a afirmativa correta.

- A) Cobertura de 100% de instruções garante que todos os defeitos relevantes foram encontrados.
- B) Cobertura mede o grau em que itens especificados foram exercitados por uma suíte de testes, mas não garante ausência de defeitos.
- C) Cobertura de ramos é sempre inferior e menos útil que cobertura de linhas.
- D) Cobertura é uma técnica de teste caixa-preta baseada exclusivamente nos requisitos funcionais.
- E) Cobertura só é aplicável a testes de sistema executados em homologação.

7. Um requisito estabelece que pedidos até R\$ 1.000,00 têm aprovação automática; pedidos acima de R\$ 1.000,00 exigem autorização gerencial. Para projetar casos de teste com maior chance de encontrar erro na fronteira dessa regra, a técnica mais adequada é

- A) teste exploratório sem critério prévio.
- B) teste de desempenho com carga crescente.
- C) análise de valor-limite.
- D) teste de usabilidade.
- E) mocking de dependências externas.

8. A respeito da pirâmide de testes em uma esteira de integração contínua, analise as afirmativas. I. Testes unitários tendem a estar na base por serem numerosos, rápidos e baratos. II. Testes de interface ponta a ponta tendem a ser menos numerosos por serem mais lentos e frágeis. III. A existência de testes ponta a ponta elimina a utilidade de testes unitários. Está correto o que se afirma em

- A) I, apenas.
- B) III, apenas.
- C) I e III, apenas.
- D) I e II, apenas.
- E) I, II e III.

9. Após uma alteração em regra de cálculo de multa, a equipe executa novamente um conjunto de testes já existente para verificar se funcionalidades anteriormente corretas continuam funcionando. A finalidade predominante dessa execução é teste

- A) de regressão.
- B) de estresse.
- C) de usabilidade.
- D) estático.
- E) de instalação.

10. Em um sistema de peticionamento eletrônico, uma suíte percorre o fluxo completo de autenticação, protocolo de documento, validação de formato, gravação, tramitação e consulta posterior, em ambiente de homologação representativo. O foco é avaliar o comportamento do produto integrado frente aos requisitos. Trata-se de teste

- A) unitário, pois cada etapa possui assertivas.
- B) de integração apenas entre duas classes.
- C) estático de requisito.
- D) de contrato isolado.
- E) de sistema.

11. Observe o pseudocódigo a seguir: se usuario.ativo e usuario.perfil == 'GESTOR', permitirAcesso(); caso contrário, negarAcesso(). Uma equipe deseja derivar testes a partir da estrutura lógica interna, buscando exercitar resultados de decisão verdadeiro e falso. A técnica é associada principalmente a

- A) partição de equivalência, pois ignora a estrutura interna.
- B) teste de aceitação do usuário, pois envolve perfil de acesso.
- C) teste caixa-branca com cobertura de decisão/ramo.
- D) teste de carga, pois verifica múltiplos perfis.
- E) teste de compatibilidade entre navegadores.

12. Uma equipe afirma que seus testes são suficientes porque todos passam em ambiente local. Entretanto, na esteira CI/CD, alguns falham de modo intermitente por dependência de relógio do sistema, ordem de execução e dados compartilhados. Esses testes são melhor caracterizados como

- A) estáticos, pois dependem de inspeção manual.
- B) flaky, pois apresentam comportamento não determinístico.
- C) de aceitação, pois dependem do usuário final.
- D) de desempenho, pois falham por lentidão sempre constante.
- E) de caixa-preta, pois não usam código-fonte.

13. Em testes de software, o oráculo de teste é melhor entendido como

- A) ferramenta de versionamento usada para comparar branches de teste.
- B) ambiente isolado onde os testes de integração são executados.
- C) métrica percentual de cobertura obtida na execução.
- D) critério ou mecanismo que permite determinar se o resultado observado está correto.

E) objeto simulado que verifica chamadas de métodos externos.

14. Assinale a alternativa que apresenta distinção adequada entre teste de carga e teste de estresse.

A) Teste de carga avalia comportamento sob volume esperado ou crescente; teste de estresse submete o sistema a condições extremas ou anormais.

B) Teste de carga é unitário; teste de estresse é sempre estático.

C) Teste de carga verifica usabilidade; teste de estresse verifica apenas ortografia de mensagens.

D) Teste de carga e teste de estresse são sinônimos perfeitos em engenharia de software.

E) Teste de estresse é usado somente para banco de dados, enquanto carga é usado somente para frontend.

15. Durante a execução de uma funcionalidade, um defeito existente em uma regra de arredondamento manifesta-se ao usuário por meio de valor monetário incorreto na tela. Nessa situação, a manifestação observável do problema corresponde a

A) erro humano.

B) caso de teste.

C) falha.

D) stub.

E) cobertura.

16. Sobre teste de sistema, assinale a afirmativa incorreta.

A) Pode avaliar requisitos funcionais e não funcionais do sistema integrado.

B) Normalmente ocorre em ambiente representativo, ainda que não seja produção.

C) Pode envolver fluxos completos que atravessam várias camadas da aplicação.

D) Pode revelar defeitos não percebidos em testes unitários e de integração restrita.

E) Dispensa totalmente os testes unitários, pois o sistema completo contém todas as unidades.

17. Um analista propõe a seguinte estratégia: identificar classes válidas e inválidas de entrada para um campo 'idade', escolhendo poucos representantes de cada classe, em vez de testar todos os valores possíveis. Essa abordagem corresponde à técnica de

A) cobertura de ramos.

B) partição de equivalência.

C) teste de estresse.

D) spy de dependências.

E) teste de mutação obrigatória.

18. No contexto de testes automatizados de API, uma suíte verifica se uma resposta HTTP retorna status 201, cabeçalho Location e corpo JSON compatível com o contrato publicado. O foco principal desse teste, no recorte descrito, é

- A) a substituição de uma dependência por dummy.
- B) a análise estática de código-fonte.
- C) a medição de usabilidade por entrevista.
- D) a verificação de contrato/comunicação de integração da API.
- E) a comprovação de ausência absoluta de defeitos.

19. Considere as seguintes afirmações sobre testes de software. I. Testes podem revelar a presença de defeitos, mas não demonstram de forma absoluta sua ausência. II. Testes bem planejados fornecem informação sobre qualidade e risco. III. A automação de testes transforma automaticamente testes de sistema em testes unitários. Está correto o que se afirma em

- A) I e II, apenas.
- B) I, apenas.
- C) II e III, apenas.
- D) III, apenas.
- E) I, II e III.

20. Uma organização mantém uma suíte mínima que roda logo após cada implantação em homologação para verificar rapidamente se as funcionalidades críticas estão de pé: autenticação, página inicial, consulta principal e criação básica de registro. Sem entrar em todos os detalhes de regra de negócio, essa suíte é mais bem descrita como teste

- A) de decisão estrutural.
- B) de valor-limite.
- C) smoke.
- D) de mutação.
- E) dummy.

14. Gabarito resumido

1-B, 2-D, 3-C, 4-A, 5-E, 6-B, 7-C, 8-D, 9-A, 10-E, 11-C, 12-B, 13-D, 14-A, 15-C, 16-E, 17-B, 18-D, 19-A, 20-C.

15. Comentários completos

Questão 1 - Gabarito: B

A correta é B. O enunciado limita o escopo a uma classe e uma regra de cálculo, sem dependências externas, com assertivas diretas. Isso caracteriza teste unitário.

Alternativa A: parece tentadora porque a regra importa ao usuário, mas o escopo executado não é o sistema completo.

Alternativa C: parece tentadora porque a regra deriva de requisito, mas não há interação entre componentes.

Alternativa D: aceitação envolve avaliação frente à necessidade de negócio pelo cliente/usuário, não apenas teste técnico isolado.

Alternativa E: regressão descreve finalidade de reexecutar testes após mudança; não é o nível descrito.

Questão 2 - Gabarito: D

A correta é D. O teste sobe aplicação, chama endpoint real, usa serialização e banco de teste. O foco é interação entre partes, logo integração.

Alternativa A: automatização e JUnit não tornam o teste unitário; o escopo é amplo.

Alternativa B: dados semelhantes aos de produção não bastam para aceitação.

Alternativa C: pode também ser usado em regressão, mas a classificação predominante do escopo é integração.

Alternativa E: teste estático não executa o software; aqui há execução real.

Questão 3 - Gabarito: C

A correta é C. I e II estão corretas. III está errada porque regressão é tipo/finalidade de teste, não nível situado entre integração e sistema.

Alternativa A: incompleta, pois II também está correta.

Alternativa B: incompleta, pois I também está correta.

Alternativa D: inclui III, que confunde tipo com nível.

Alternativa E: inclui a afirmação III, tecnicamente incorreta.

Questão 4 - Gabarito: A

A correta é A: Fake, Stub, Mock, Spy correspondem à sequência 3 - 1 - 2 - 4.

Alternativa B: troca fake por stub no início; stub responde, fake implementa simplificada.

Alternativa C: inverte mock e fake/spy.

Alternativa D: coloca spy como implementação simplificada, o que é incorreto.

Alternativa E: troca stub e mock: mock valida interação; stub devolve resposta pré-definida.

Questão 5 - Gabarito: E

A correta é E. O teste envolve classe, template, injeção e interação simulada; por isso tem caráter de integração de componente, ainda que controlado.

Alternativa A: não é estático, pois há execução do componente.

Alternativa B: é tentadora, mas o enunciado não testa apenas a classe isolada.

Alternativa C: não há volume, concorrência ou desempenho.

Alternativa D: não percorre o sistema inteiro com backend real e ambiente representativo.

Questão 6 - Gabarito: B

A correta é B. Cobertura indica grau de exercício de itens por uma suíte, mas não garante ausência de defeitos nem qualidade semântica.

Alternativa A: é a pegadinha clássica: executar linhas não prova que as assertivas capturam todos os problemas.

Alternativa C: não há hierarquia fixa; cobertura de ramos pode ser mais rigorosa que linhas em muitos casos.

Alternativa D: cobertura é associada sobretudo a critérios estruturais, não exclusivamente caixa-preta.

Alternativa E: também se aplica a testes unitários e de integração.

Questão 7 - Gabarito: C

A correta é C. Quando a regra muda em torno de uma fronteira, como 1.000,00, a análise de valor-limite é técnica de alto rendimento.

Alternativa A: exploratório pode encontrar defeitos, mas não é a técnica sistemática indicada.

Alternativa B: carga avalia volume/desempenho, não fronteira de regra.

Alternativa D: usabilidade avalia experiência de uso.

Alternativa E: mocking é mecanismo de isolamento, não técnica de seleção de valores.

Questão 8 - Gabarito: D

A correta é D. I e II expressam a lógica da pirâmide. III é falsa: testes ponta a ponta não eliminam testes unitários.

Alternativa A: incompleta, pois II também está correta.

Alternativa B: III está errada.

Alternativa C: inclui III e omite II.

Alternativa E: inclui III, que absolutiza indevidamente o papel dos testes E2E.

Questão 9 - Gabarito: A

A correta é A. Reexecutar testes após modificação para detectar quebras em funcionalidades já existentes é finalidade de regressão.

Alternativa B: estresse envolve condições extremas.

Alternativa C: usabilidade mede facilidade de uso.

Alternativa D: teste estático não executa o software.

Alternativa E: instalação verifica implantação/instalação, não comportamento preservado após mudança.

Questão 10 - Gabarito: E

A correta é E. O cenário percorre fluxo completo em ambiente representativo, avaliando o produto integrado; isso é teste de sistema.

Alternativa A: assertivas não tornam o teste unitário.

Alternativa B: o escopo é maior que duas classes.

Alternativa C: há execução, não apenas revisão de requisito.

Alternativa D: contrato isolado de API é menor que o fluxo completo descrito.

Questão 11 - Gabarito: C

A correta é C. O teste é derivado da estrutura lógica interna e busca exercitar decisões, típico de caixa-branca com cobertura de decisão/ramo.

Alternativa A: partição de equivalência é caixa-preta e deriva da especificação.

Alternativa B: perfil de acesso não implica aceitação.

Alternativa D: não há carga, volume ou desempenho.

Alternativa E: não envolve navegadores ou plataformas.

Questão 12 - Gabarito: B

A correta é B. Testes que falham intermitentemente por relógio, ordem ou dados compartilhados são flaky, isto é, não determinísticos.

Alternativa A: são testes executáveis, não inspeções estáticas.

Alternativa C: não dependem da aceitação pelo usuário.

Alternativa D: a falha não é descrita como lentidão constante.

Alternativa E: caixa-preta/caixa-branca não explica a intermitência.

Questão 13 - Gabarito: D

A correta é D. Oráculo de teste é o critério ou mecanismo usado para decidir se o resultado observado está certo.

Alternativa A: versionamento não é oráculo.

Alternativa B: ambiente de execução não define resultado esperado.

Alternativa C: cobertura é métrica de exercício, não critério de correção.

Alternativa E: mock pode ajudar em testes, mas não é a definição de oráculo.

Questão 14 - Gabarito: A

A correta é A. Carga examina comportamento sob volume esperado/crescente; estresse empurra o sistema para condições extremas/anormais.

Alternativa B: não são definidos por nível unitário/estático.

Alternativa C: usabilidade e ortografia não são o foco desses testes.

Alternativa D: não são sinônimos perfeitos.

Alternativa E: a restrição por camada técnica é inventada.

Questão 15 - Gabarito: C

A correta é C. A falha é a manifestação observável de um defeito durante execução, como valor incorreto exibido ao usuário.

Alternativa A: erro humano pode ter causado o defeito, mas não é a manifestação em execução.

Alternativa B: caso de teste é artefato de verificação.

Alternativa D: stub é test double.

Alternativa E: cobertura é métrica.

Questão 16 - Gabarito: E

A correta é E, pois a questão pede a incorreta. Teste de sistema não dispensa totalmente testes unitários; os níveis são complementares.

Alternativa A: correta: sistema pode avaliar requisitos funcionais e não funcionais.

Alternativa B: correta: ambiente representativo é típico.

Alternativa C: correta: fluxos completos atravessam camadas.

Alternativa D: correta: defeitos emergentes podem aparecer apenas no sistema integrado.

Questão 17 - Gabarito: B

A correta é B. Dividir entradas em classes válidas e inválidas e escolher representantes é partição de equivalência.

Alternativa A: cobertura de ramos deriva da estrutura interna.

Alternativa C: estresse trata condições extremas de operação.

Alternativa D: spy é test double.

Alternativa E: teste de mutação não é obrigatório nem descrito no cenário.

Questão 18 - Gabarito: D

A correta é D. Status HTTP, cabeçalho, corpo JSON e contrato de API apontam para verificação de comunicação/contrato em integração.

Alternativa A: dummy é objeto passado sem uso relevante.

Alternativa B: há execução de API, não análise estática.

Alternativa C: não envolve estudo de usabilidade.

Alternativa E: nenhum teste comprova ausência absoluta de defeitos.

Questão 19 - Gabarito: A

A correta é A. I e II estão corretas. III é falsa: automação é modo de execução e não altera o nível conceitual do teste.

Alternativa B: incompleta, pois II também está correta.

Alternativa C: inclui III e omite I.

Alternativa D: III está errada.

Alternativa E: inclui a afirmação falsa sobre automação transformar nível de teste.

Questão 20 - Gabarito: C

A correta é C. Smoke test é uma verificação rápida de funcionalidades críticas para saber se a build/implantação está minimamente operacional.

Alternativa A: decisão estrutural é técnica caixa-branca.

Alternativa B: valor-limite seleciona entradas de fronteira.

Alternativa D: mutação altera código para avaliar capacidade dos testes de detectar mudanças.

Alternativa E: dummy é test double sem uso relevante.

16. Checklist final de memorização

- Unitário: unidade isolada, rápido, determinístico, dependências controladas.
- Integração: comunicação entre componentes, camadas, banco, API, fila ou serviços.
- Sistema: produto integrado em ambiente representativo, fluxo completo e requisitos globais.
- Regressão é finalidade/tipo, não nível.
- Carga, estresse, segurança, usabilidade e desempenho são tipos/objetivos, não níveis centrais do edital.
- Cobertura mede execução, não qualidade absoluta.
- Verificação: construir corretamente; validação: construir a coisa certa.
- Falha é observável; defeito está no artefato; erro é humano.
- Stub responde; mock verifica interação; fake implementa simplificado; spy observa; dummy preenche parâmetro.

17. Fontes usadas na calibração

- Edital retificado TJSC 2026 - Analista de Sistemas, disciplina Engenharia de Software e Desenvolvimento.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- Provas oficiais FGV correlatas de Analista/Desenvolvedor de Sistemas e ATI, usadas apenas para calibrar estilo, sem reprodução literal de questões.
- ISTQB Glossary - definições de unit testing, integration testing, coverage, test level e test type.
- ISO/IEC/IEEE 29119-1:2022 e 29119-2:2013 - conceitos gerais e processos de teste de software.
- JUnit 5 User Guide, Quarkus Testing Guide e Angular Testing Guide - referências técnicas para exemplos práticos.
- Martin Fowler - Test Pyramid, como referência secundária para organização econômica de testes automatizados.